



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences



ABSCHLUSSVORTRAG · LITERATUR-SEMINAR

Automatisierte Sicherheitstests in CI/CD-Pipelines

DevSecOps als Ansatz für sichere Softwareentwicklung

Firas Massoussi

Cyber Security & Privacy · Prof. Dr. Hackelöer

Vortrag am 16.07.2026

Motivation: Warum das Thema wichtig ist



Software wird immer schneller ausgeliefert

Agile Teams nutzen DevOps und CI/CD-Pipelines: bauen, testen und deployen in kurzen Abständen.



Sicherheit wird oft zu spät geprüft

Wird Cybersecurity erst am Ende geprüft, werden Schwachstellen spät und teuer entdeckt.



Große Angriffsfläche durch fremde Komponenten

Open-Source-Bibliotheken, Container-Images und versehentlich veröffentlichte Zugangsdaten erhöhen das Risiko.



Kernidee

Bei schneller Entwicklung darf Sicherheit kein einzelner Schritt am Ende sein.

Sie muss früh, regelmäßig und automatisiert mitlaufen. (vgl. NIST 2022, OWASP)



FORSCHUNGSFRAGE

Wie können **automatisierte Sicherheitstests in CI/CD-Pipelines** dazu beitragen, Sicherheitsprobleme **frühzeitig** zu erkennen – und welche **Vorteile und Herausforderungen** ergeben sich daraus für das Software Engineering?

Was ist DevSecOps – und wie unterscheidet es sich von DevOps?

Welche Testart passt an welche Stelle der Pipeline?

Was bringt der Ansatz – und wo sind die Grenzen?

Von DevOps zu DevSecOps – „Shift Left“

DevOps

- Verbindet Entwicklung + Betrieb
- Automatisierte Builds, Tests, CI/CD
- Schneller – aber Sicherheit läuft oft nebenher

DevSecOps

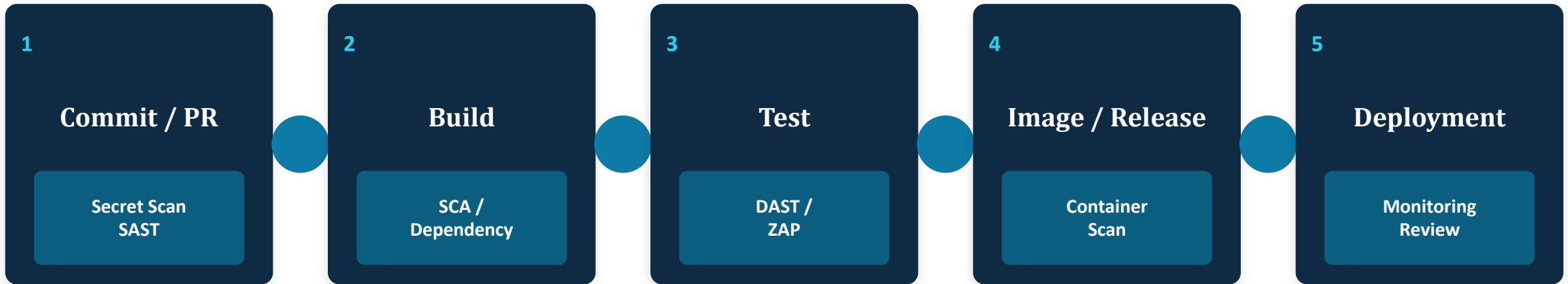
- Ergänzt DevOps um Security
- Sicherheit von Anfang an mitgedacht
- Technischer UND organisatorischer Ansatz



Shift Left Security

Sicherheitsprüfungen wandern nach „links“ – also früher in den Entwicklungsprozess. Klassische Sicherheitsmethoden sind für das Tempo von DevOps oft zu spät oder zu langsam. (vgl. Myrbakken & Colomo-Palacios 2017; Rajapakse et al. 2022; Zhao et al. 2024)

Die DevSecOps-Pipeline: Prüfungen an jeder Stelle



Eigene Darstellung auf Grundlage von NIST (2017, 2022), OWASP und GitHub Docs.



Jede Phase kann eigene Sicherheitsprüfungen enthalten. Und: Die Pipeline selbst ist ein Angriffsziel – sie hat Zugriff auf Code, Secrets und Deployment. Auch sie muss geschützt werden. (vgl. OWASP CI/CD Security)

Fünf automatisierte Testarten

Jede prüft ein anderes Objekt – deshalb ergänzen sie sich, statt sich zu ersetzen.



SAST

Static App Security Testing

Prüft ruhenden Quellcode – ohne die App auszuführen.



DAST

Dynamic App Security Testing

Testet die laufende Anwendung von außen (z. B. ZAP).



SCA

Dependency Scanning

Prüft externe Bibliotheken auf bekannte Schwachstellen.



Secret Scanning

Zugangsdaten

Findet API-Keys, Tokens & Passwörter im Code / in der Git-Historie.



Container Scanning

Images

Prüft Container-Images auf verwundbare Pakete vor dem Release.

Vergleich: Wer prüft was – und wann?

Testart	Pipeline-Phase	Prüfobjekt	Typische Funde
SAST	Commit / PR / Build	Quellcode	Unsichere Muster, Codefehler
DAST	Testumgebung	Laufende Anwendung	Unsicheres Verhalten, Eingaben
SCA / Dependencies	Build	Externe Bibliotheken	Bekannte CVEs in Paketen
Secret Scanning	Commit / PR	Code & Git-Historie	API-Keys, Tokens, Passwörter
Container Scanning	Vor Deployment	Container-Image	Verwundbare Systempakete



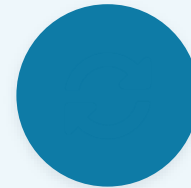
Kernaussage: Kein einzelner Test deckt alles ab – erst die Kombination ergibt eine sichere Pipeline.

Vorteile von DevSecOps



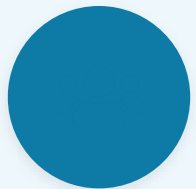
Frühere Erkennung

Probleme werden nah an ihrer Entstehung sichtbar – Fehler sind günstiger zu beheben.



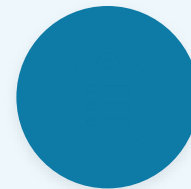
Regelmäßige Prüfungen

Tests laufen bei jeder Änderung automatisch, nicht nur einmal am Ende.



Bessere Zusammenarbeit

Entwicklung, Betrieb & Security teilen gemeinsame Regeln und Ziele.



Nachvollziehbarkeit

Pipeline-Berichte dokumentieren Prüfungen – hilfreich für Audits und Qualität.

Herausforderungen von DevSecOps



Fehlalarme (False Positives)

Zu viele unkritische Meldungen führen dazu, dass echte Warnungen übersehen werden.



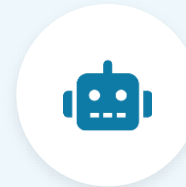
Aufwand & Tool-Auswahl

Pipeline anpassen, Tools einrichten und Regeln festlegen kostet Zeit und Pflege.



Fehlendes Security-Wissen

Ein Team muss verstehen, was eine Meldung bedeutet, um richtig zu reagieren.



Zu starkes Vertrauen in Tools

Automatisierung ersetzt keine Architektur-Reviews und Bedrohungsmodelle.



FAZIT & AUSBLICK

Automatisierte Tests machen Sicherheitsprobleme früher sichtbar wenn sie über die Pipeline verteilt werden.

Früh: Secret Scanning & SAST → Build: Dependency Scanning → Test: DAST → vor Deployment: Container Scanning

Nutzen entsteht nicht durch möglichst viele Tools, **sondern durch die Verbindung von Automatisierung, klaren Prozessen und Zusammenarbeit.**

Vielen Dank für Ihre Aufmerksamkeit ich freue mich auf Ihre Fragen.